

JAVA TECHNICAL TEST INSTRUCTIONS – SPRINGBOOT

REST API for sales management in a supermarket chain

OBJECTIVE

The purpose of this test is to evaluate knowledge of **Java + Spring Boot**, including the development of a complete **RESTful API** that implements **CRUD** operations with **JPA**, entity relationships, error and exception handling, use of DTOs, REST best practices, and functional programming (use of Lambdas and Streams) where applicable.

CASE DESCRIPTION

A well-known supermarket chain wants to digitize its sales control system. To do this, it needs an API that allows it to (at a basic level):

- Register products with their respective prices.
- Manage the branches where the products are sold.
- Record sales made in a branch, specifying the products sold in quantities.

The company then wants to view sales by branch, total revenue, filter best-selling products, etc.

MAIN ENTITIES

- **Branch**: represents a physical location of the supermarket (one for each location).
- **Product**: represents an item that can be sold (e.g., rice, bottle of water, etc.).
- **Sale**: contains one or more product lines associated with a branch.

RELATIONSHIPS

- A **branch** can have **many sales**.
- A **sale** has **many associated products**.
- The **same product** can be in **many sales**.

TECHNICAL REQUIREMENTS

- Use **Spring Boot** with **JPA** for database management.
- Relational database (e.g., H2 or MySQL).
- Expose **RESTful** endpoints to perform CRUDs (GET, POST, PUT, DELETE, or whatever methods are deemed necessary).
- Use **DTOs** to separate domain models and external representation.
- Proper error handling with **ResponseEntity**, correct HTTP codes (status code), and clear messages.
- Use of **lambdas** or **streams** in at **least one backend operation**.
- Modular organization of the project (service, repository, controller).

USER STORIES (FUNCTIONAL REQUIREMENTS)

PRODUCTS

1. Obtain product listing

- Method: GET
- Path: /api/products
- Description: List all registered products.

2. Register new product

- Method: POST
- Path: /api/products
- Description: Create a product with name, price, and category.

3. Update existing product

- Method: PUT
- Path: /api/products/{id}
- Description: Modify the data for a specific product.

4. Delete a product

- Method: DELETE
- Path: /api/products/{id}
- Description: Remove a product from the system.

BRANCHES

1. Get a list of branches

- Method: GET
- Path: /api/branches
- Description: List all branches in the system.

2. Register new branch

- Method: POST
- Path: /api/branches
- Description: Create a new branch with address, name, etc.

3. Update existing branch

- Method: PUT
- Path: /api/branches/{id}
- Description: Modify the data of an existing branch.

4. Delete a branch

- Method: DELETE
- Path: /api/branches/{id}
- Description: Delete a branch from the system.

SALES

1. Register new sale

- Method: POST
- Path: /api/sales
- Payload

```
{
  "branchId": 1,
  "detail": [
    {"productId": 10, "quantity": 2},
    {"productId": 5, "quantity": 1},
  ]
}
```

- Description: Create a new sale for a branch with products and quantities.

2. Get sales by branch and date

- Method: GET
- Path: /api/sales?branchId=1&date=2025-06-01
- Description: List sales made on a specific date for a branch.

3. Delete/Cancel a sale

- Method: DELETE
- Path: /api/sales/{id}
- Description: Delete a registered sale.
- Use of logical deletion will be evaluated.

Sales CANNOT BE MODIFIED without superuser permissions (no need to implement this).

EXTRA – STATISTICS (Optional, not mandatory)

1. Get the best-selling product

- Method: GET
- Path: /api/statistics/best-selling-product
- Description: Calculate the best-selling product using Java Streams.

INSTRUCCIONES PRUEBA TÉCNICA

JAVA – SPRINGBOOT

API REST para la gestión de ventas en una cadena de supermercados

OBJETIVO

El objetivo de esta prueba es evaluar conocimientos en **Java + Spring Boot**, incluyendo el desarrollo de una **API RESTful** completa que implemente operaciones CRUD con JPA, relaciones entre entidades, control de errores y excepciones, uso de DTOs, buenas prácticas REST y programación funcional (uso de Lambdas y Streams) donde aplique.

DESCRIPCIÓN DEL CASO

Una reconocida cadena de supermercados desea digitalizar su sistema de control de ventas. Para ello necesita una API que permita (de forma básica):

- Registrar productos con sus respectivos precios.
- Gestionar las sucursales donde se venden los productos.
- Registrar ventas realizadas en una sucursal, especificando los productos vendidos en cantidades.

La empresa desea consultar luego las ventas por sucursal, totalizar ingresos, filtrar productos más vendidos, etc.

ENTIDADES PRINCIPALES

- **Sucursal**: representa una ubicación física del supermercado (una por cada ubicación)
- **Producto**: representa un artículo que puede venderse (ejemplo arroz, botella de agua, etc).
- **Venta**: contiene una o más líneas de productos, asociadas a una sucursal.

RELACIONES

- Una **Sucursal** puede tener **muchas ventas**.
- Una **Venta** tiene **muchos productos asociados**.
- Un mismo **Producto** puede estar en **muchas ventas**.

REQUISITOS TÉCNICOS

- Utilizar **Spring Boot** con JPA para manejo de base de datos.
- Base de datos relacional (por ejemplo H2 o MySQL).
- Exponer endpoints RESTful para realizar CRUDs (GET, POST, PUT, DELETE o los métodos que se consideren necesarios).
- Utilizar **DTOs** para separar modelos de dominio y representación externa.
- Manejo adecuado de errores con **ResponseEntity**, códigos HTTP correctos (status code) y mensajes claros.
- Uso de **lambdas** o **streams** en al menos **una operación del backend**.
- Organización modular del proyecto (service, repository, controller).

HISTORIAS DE USUARIO (REQUERIMIENTOS FUNCIONALES) PRODUCTOS

1. Obtener listado de productos

- Método: GET
- Path: /api/productos
- Descripción: Listar todos los productos registrados.

2. Registrar nuevo producto

- Método: POST
- Path: /api/productos
- Descripción: Crear un producto con nombre, precio y categoría.

3. Actualizar producto existente

- Método: PUT
- Path: /api/productos/{id}
- Descripción: Modificar los datos de un producto específico.

4. Eliminar un producto

- Método: DELETE
- Path: /api/productos/{id}
- Descripción: Eliminar un producto del sistema.

SUCURSALES

1. Obtener listado de sucursales

- Método: GET
- Path: /api/sucursales
- Descripción: Listar todas las sucursales del sistema.

2. Registrar nueva sucursal

- Método: POST
- Path: /api/sucursales
- Descripción: Crear una nueva sucursal con dirección, nombre, etc.

3. Actualizar sucursal existente

- Método: PUT
- Path: /api/sucursales/{id}
- Descripción: Modificar los datos de una sucursal existente.

4. Eliminar una sucursal

- Método: DELETE
- Path: /api/sucursales/{id}
- Descripción: Eliminar una sucursal del sistema.

VENTAS

1. Registrar nueva venta

- Método: POST
- Path: /api/ventas
- Payload

```
{
  "sucursalId": 1,
  "detalle": [
    {"productid": 10, "cantidad": 2},
    {"productid": 5, "cantidad": 1},
  ]
}
```

- Descripción: Crear una nueva venta para una sucursal con productos y cantidades

2. Obtener ventas por sucursal y fecha

- Método: GET
- Path: /api/ventas?sucursalId=1&fecha=2025-06-01
- Descripción: Listar ventas realizadas en una fecha específica para una sucursal.

3. Eliminar / Anular una venta

- Método: DELETE
- Path: /api/ventas/{id}
- Descripción: Eliminar una venta registrada.
- Se valorará uso de borrado lógico

Las ventas NO SE PUEDEN MODIFICAR sin permisos de superusuario (no es necesario implementar esto).

EXTRA – ESTADÍSTICAS (Opcional no Obligatorio)

1. Obtener el producto más vendido

- Método: GET
- Path: /api/estadisticas/producto-mas-vendido
- Descripción: Calcular el producto más vendido utilizando Java Streams.